

Constructor and types

Absolutely. For C++ OOP interviews, constructors are **very commonly asked**, especially differences between default, parameterized, copy, and delegating constructors.

1. What is a Constructor?

A **constructor** is a special member function that is automatically called when an object is created.

Key properties:

- Same name as the class
- No return type, not even `void`
- Called automatically
- Used to initialize an object
- Can be overloaded

```
class Student {
public:
    Student() {
        cout << "Constructor called";
    }
};

int main() {
    Student s;    // Constructor automatically called
}
```

Mental model:

Constructor = how an object gets initialized when it comes into existence.

2. Default Constructor

A constructor that can be called **without arguments**.

```
class Student {
public:
    Student() {
        cout << "Default constructor\n";
    }
};

int main() {
    Student s;
}
```

Output:

Default constructor

Important interview point

A default constructor can be:

```
Student();
```

or a constructor with all parameters having default values:

```
Student(int age = 18);
```

Both can be called as:

```
Student s;
```

Compiler-generated default constructor

If you don't write **any constructor**, C++ may provide a default constructor for you.

```
class Student {
    int age;
};

int main() {
    Student s;    // OK
}
```

But be careful:

If you define another constructor:

```
class Student {  
public:  
    Student(int age) {}  
};  
  
Student s; // ✖ Error
```

The compiler **doesn't automatically provide a no-argument constructor** once you declare a constructor yourself.

Interview question

Q: Does the compiler always provide a default constructor?

Answer: No.

It is generated only when appropriate, and declaring constructors yourself can prevent implicit default-constructor generation.

3. Parameterized Constructor

A constructor that accepts arguments.

```
class Student {  
    int age;  
    string name;  
  
public:  
    Student(int a, string n) {  
        age = a;  
        name = n;  
    }  
};  
  
int main() {  
    Student s(20, "Lalit");  
}
```

Here:

```
Student s(20, "Lalit");
```

calls:

```
Student(int, string)
```

Better way: Member Initializer List

In C++, you should generally initialize members like this:

```
class Student {  
    int age;  
    string name;  
  
public:  
    Student(int a, string n)  
        : age(a), name(n) {}  
};
```

rather than:

```
Student(int a, string n) {  
    age = a;  
    name = n;  
}
```

Why?

Because:

```
: age(a), name(n)
```

directly initializes the members.

Whereas:

```
age = a;
```

first creates/initializes the member and then assigns to it.

Very important interview point

Some members **must** be initialized using the initializer list:

- `const` data members
- references
- members without a default constructor
- base classes

Example:

```
class Student {  
    const int age;  
  
public:  
    Student(int a) : age(a) {}  
};
```

This would be invalid:

```
Student(int a) {  
    age = a; // ✗  
}
```

because `const` members cannot be assigned after initialization.

4. Copy Constructor

This one is **very important for interviews**.

A copy constructor creates a new object by copying an existing object.

General form:

```
ClassName(const ClassName& other);
```

Example:

```
class Student {  
public:  
    int age;  
  
    Student(int a) : age(a) {}  
  
    Student(const Student& other) {  
        age = other.age;  
    }  
};  
  
int main() {  
    Student s1(20);  
  
    Student s2(s1);  
}
```

Here:

```
Student s2(s1);
```

calls the copy constructor.

Why `const ClassName&?`

This:

```
Student(const Student& other)
```

is preferred because:

Reference

If you wrote:

```
Student(Student other)
```

then passing `other` itself requires copying `other` → which requires calling the copy constructor → which requires copying again → potentially infinite recursion.

So we use a reference:

```
const Student& other
```

`const`

Because we don't want to modify the source object.

5. When is Copy Constructor Called?

This is an **excellent interview question**.

Case 1 — Explicit object initialization

```
Student s1(20);  
Student s2(s1);
```

Copy constructor called.

Case 2 — Copy initialization

```
Student s2 = s1;
```

Copy constructor is involved.

Notice:

```
Student s2 = s1;
```

does **not** mean assignment.

It's initialization of a new object.

Case 3 — Passing object by value

```
void print(Student s) {}  
  
Student s1(20);  
print(s1);
```

A copy may be made to create the parameter `s`.

Case 4 — Returning object by value

```
Student createStudent() {  
    Student s(20);  
    return s;  
}
```

A copy/move may conceptually be involved, although modern C++ often eliminates it through **copy elision**.

6. Copy Constructor vs Assignment Operator

This is one of the **most important distinctions**.

Copy constructor

Creates a **new object**.

```
Student s2 = s1;
```

Think:

```
"Create s2 using s1."
```

Copy assignment

Both objects already exist.

```
Student s2;  
s2 = s1;
```

Think:

```
"s2 already exists. Replace its contents with s1's."
```

So:

```
Student s2 = s1;
      ↑
copy construction
```

versus:

```
Student s2;
s2 = s1;
  ↑
copy assignment
```

Interview trap

Q: Is `Student s2 = s1` assignment?

No.

It is **copy initialization**, which invokes the copy constructor (subject to copy-elision/other initialization rules).

7. Shallow Copy vs Deep Copy

This becomes important when your class owns dynamically allocated memory.

```
class Student {
    int* age;

public:
    Student(int a) {
        age = new int(a);
    }
};
```

Suppose:

```
Student s1(20);
Student s2 = s1;
```

The compiler-generated copy constructor performs a **member-wise copy**.

So conceptually:

```
s1.age —————> [20]
                ▲
s2.age —————┘
```

Both pointers point to the same memory.

That's **shallow copy**.

This can cause problems such as:

- double deletion
- dangling pointers
- unintended shared ownership

A deep copy would create separate memory:

```
s1.age —————> [20]
s2.age —————> [20]
```

Example:

```
Student(const Student& other) {
    age = new int(*other.age);
}
```

Now each object owns its own integer.

8. Delegating Constructor

A **delegating constructor** calls another constructor of the same class.

Introduced in **C++11**.

Example:

```
class Student {
    int age;
    string name;

public:
    Student() : Student(18, "Unknown") {}

    Student(int a, string n)
        : age(a), name(n) {}
};
```

Now:

```
Student s;
```

calls:

```
Student()
↓
Student(18, "Unknown")
```

The first constructor **delegates** initialization to the second.

Why use it?

To avoid duplicate initialization logic.

Without delegation:

```
Student() {
    age = 18;
    name = "Unknown";
}

Student(int a, string n) {
    age = a;
    name = n;
}
```

You have duplicated logic.

With delegation:

```
Student() : Student(18, "Unknown") {}

Student(int a, string n)
    : age(a), name(n) {}
```

Only one constructor contains the actual initialization logic.

Mental model

Delegating constructor = one constructor forwards the job to another constructor in the same class.

9. Can Constructors Be Overloaded?

Yes.

```
class Student {
public:
    Student() {}

    Student(int age) {}

    Student(int age, string name) {}

    Student(const Student& other) {}
};
```

These are different constructors based on their parameter lists.

This is **constructor overloading**.

10. The Four You Should Know

ConstructorPurposeExampleDefaultCreate object without arguments`Student s;`ParameterizedInitialize using provided values`Student s(20);`CopyCreate object from another object`Student s2(s1);`DelegatingOne constructor calls another constructor`Student() : Student(18) {}`

Interview Questions You Should Be Able to Answer

Basic

1. What is a constructor?

Special member function automatically called when an object is initialized, used to initialize the object's state.

2. Can a constructor have a return type?

No.

```
Student() {}           // ☒
void Student() {}      // ☐
```

3. Can constructors be overloaded?

Yes.

4. Can constructors be `virtual`?

No.

Reason: virtual dispatch requires an already-existing object/vtable relationship, while a constructor is responsible for creating/initializing the object.

5. Can a constructor be private?

Yes.

Common uses include controlled object creation, factory patterns, and Singleton-style designs.

Medium

6. What happens if you don't define a constructor?

The compiler may implicitly generate a default constructor, provided the conditions for implicit generation are satisfied.

7. What is a copy constructor?

A constructor that initializes a new object from an existing object of the same class.

```
Student(const Student& other);
```

8. Copy constructor vs copy assignment?

```
Student b(a); // copy constructor
b = a;        // copy assignment
```

9. Why does a copy constructor usually take `const T&`?

To avoid recursive copying and allow copying from const objects without modifying the source.

10. What is a delegating constructor?

A constructor that delegates initialization to another constructor of the same class.

```
Student() : Student(18) {}
```

★ Interview Trap: Constructor Initialization Order

Suppose:

```
class A {
    int x;
    int y;

public:
    A() : y(20), x(y) {}
};
```

What happens?

x is initialized before **y**, despite the initializer list saying **y** first.

Members are initialized in the **order they're declared in the class**:

```
int x; // first
int y; // second
```

So this effectively happens:

```
x(y); // y isn't initialized yet ✗  
y(20);
```

Rule to memorize:

Member initialization order = declaration order, NOT initializer-list order.

This is a very common C++ interview question.

Final Mental Model

Think of constructors as different ways of bringing an object into existence:

```
Student s;  
↓  
Default constructor  
  
Student s(20);  
↓  
Parameterized constructor  
  
Student s2(s1);  
↓  
Copy constructor  
  
Student()  
↓  
Delegating constructor  
↓  
Student(18, "Unknown")
```

And remember these **5 interview distinctions** especially well:

```
Constructor  
↓  
Initialization  
  
Copy constructor  
↓  
Creates NEW object from existing object  
  
Copy assignment  
↓  
Copies into EXISTING object  
  
Delegating constructor  
↓  
Constructor calls another constructor of SAME class  
  
Initializer list  
↓  
Direct member/base initialization
```

If you're preparing specifically for OOP interviews, **copy constructor + copy assignment + destructor + Rule of 3/5/0** is the next natural topic to learn after these.